TECHNICAL PORTFOLIO

PORTFOLIO

최동욱 (QA Engineering)

데이터 흐름을 이해하고 근거로 결함을 분석하는 QA 엔지니어

Email choi.dongwook20@gmail.com
GitHub github.com/DWinging
Velog velog.io/@dong20

CONTENTS

프로젝트 구현 과정과 문제 해결 경험을 중심으로 정리한 기술 포트폴리오입니다.

01 **GYUL**
AI 기반 레벨별 테크 큐레이션 서비스의 QA 수행 경험

02 **프롬프트 작성 실력 향상 프로그램**
기능 명세를 바탕으로 테스트를 설계하고 검증 결과와 결함 이력을 관리한 경험

03 **알고리즘 문제 해결 기반의 코드 최적화 경험**
문제 해결을 넘어, 실행 성능과 코드 최적화 관점으로 확장한 경험

Quality Assurance | SSAFY 서울 7반 최우수상 | Completed

01. AI 기반 레벨별 테크 큐레이션 서비스 : 결(GYUL)

QA 전담으로 테스트 설계부터 결함 공유·재검증까지 수행

프로젝트명	결(GYUL)	**개발 기간**	2026.05.22 ~ 2026.06.25
개발 인원	3인 (FE 1 · BE 1 · QA 1)	**담당 역할**	QA
핵심 결과	테스트 케이스 206건 전건 수행 · 유효 성공률 98.9% · 서울 7반 최우수상		

프로젝트와 QA 범위

결(GYUL)은 하나의 기술 아티클을 **사용자 수준(Lv1~Lv4)**에 맞게 재구성하고, 학습 관리와 기술 토론 기능을 제공하는 **AI 기반 테크 큐레이션 서비스**입니다.

QA를 담당하여 회원가입, 로그인, 아티클, 컬렉션, 아레나, 구독, 알림, 프로필 등 **13개 기능 도메인**을 구분하고, 사용자 시나리오 기반의 정상·예외·경계값 테스트를 설계했습니다.

수행한 QA 업무

테스트 설계와 문서화

- 실제 사용자 흐름을 기준으로 기능별 테스트 범위 정의
- 테스트 케이스 **206건** 작성 및 전건 수행
- 입력값, 예상 결과, 실제 결과, 상태, 결함 로그를 Google Sheet에 기록
- 검증 결과를 날짜별로 누적해 이전 결과와 수정 이력 추적

결함 공유와 재검증

- 발견한 결함의 재현 조건과 실제 결과를 GitHub Issue로 공유
- 연결된 PR에서 변경 내용을 확인한 뒤 동일 시나리오 재수행
- 새로운 문제가 발생하면 검증 근거를 추가해 Issue Re-open
- 수정된 기능과 연결된 사용자 흐름의 사이드 이펙트 확인

협업 기반 결함 관리 흐름

01 시나리오 검증
정상·예외·경계값 조건으로 기능 동작 확인

→

02 근거 기록
실제 결과와 재현 순서를 테스트 문서에 기록

→

03 Issue 공유
기대 결과와 실제 결과를 개발 담당자에게 전달

→

04 수정 확인
연결된 PR과 변경 내용을 확인하고 재테스트 준비

→

05 재검증
동일 시나리오 재수행, 문제 발생 시 Re-open

협업 방식: 결함을 단순히 전달하는 데 그치지 않고, 개발자가 동일한 상황을 확인할 수 있도록 재현 조건과 검증 결과를 함께 공유했습니다. 수정 이후에는 동일 시나리오와 연결 기능을 다시 확인했습니다.

Validation Case | QA Retrospective

대표 검증 사례와 QA 회고

비즈니스 로직 결함을 발견하고 수정 결과까지 재검증한 경험

대표 검증 사례

Arena 베팅 포인트 정산 오류

문제 발견

베팅 결과가 아닌 **찬성·반대 참여 인원 비율**에 따라 포인트가 분배되는 오류를 발견했습니다.

공유 및 재검증

잘못된 계산 로직과 재현 조건을 GitHub Issue로 공유하고, 수정 후 동일 시나리오를 재수행해 정상 동작을 확인했습니다.

테스트 운영 결과

도메인 (시트명)	주요 검증 내용	테스트케이스 수	Pass	Fail	진행 중	Block	미수행
Auth_회원가입	이름/이메일/비밀번호 유효성 체크	6	6	0	0	0	0
Onboarding_온보딩	관심사 다중 선택 전송 및 유실 여부 체크	15	15	0	0	0	0
Login_로그인	정상 로그인 / 로그아웃 여부 체크	7	4	0	0	3	0
Article_아티클	아티클 목록 조회, 상세 조회 테스트 댓글 CRUD 통합 테스트	28	26	0	0	2	0
Article_아티클_작성	아티클 작성, 수정, 삭제 기능 테스트	23	23	0	0	0	0
Arena_아레나	아레나 핵심 비즈니스 로직 체크	25	21	0	0	4	0
Collection_컬렉션	컬렉션 핵심 비즈니스 로직 체크	26	25	0	0	1	0
Notification_알림	알림 기능 테스트	13	11	0	0	2	0
Subscribe_구독	구독 기능 테스트	3	3	0	0	0	0
Profile_프로필	프로필 페이지 기능 테스트	39	28	2	0	9	0
Search_검색	검색 기능 테스트	7	7	0	0	0	0
UI/UX	UI/UX 개선 사항	4	4	0	0	0	0
ETC_기타	시스템 전체 오류, 기능 공통 오류 통합 테스트	10	6	0	0	4	0
합계 / 집계		206	179	2	0	25	0

206건
테스트 케이스
13개 기능 도메인 대상 설계 및 수행

100%
테스트 수행률
계획한 테스트 케이스 전건 수행

98.9%
유효 성공률
Block 제외 181건 중 179건 Pass

느낀 점과 한계

- 구체적인 기능 명세가 부족해 구현 결과를 기준으로 테스트를 설계하면서, 요구사항이 변경될 때마다 **검증 기준도 함께 수정**해야 했습니다.
- Google Sheet에 테스트 결과와 결함 로그를 누적했지만, 전체 진행 상황과 테스트 케이스별 결함 상태를 **한눈에 추적하기 어려운 한계**가 있었습니다.
- 로그인·회원가입 시나리오를 Playwright로 자동화했지만 기능 변경에 따라 테스트 코드 수정이 반복되어, 자동화 전에 **안정된 검증 기준과 반복 가치가 있는 범위**를 선정해야 한다는 점을 배웠습니다.

참고 자료

📊 Test Case Sheet — Google Sheets

Quality Assurance | TestRail · Jira | In Progress

02. 프롬프트 작성 실력 향상 프로그램

기능 명세를 바탕으로 테스트를 설계하고 검증 결과와 결함 이력을 관리

프로젝트명	프롬프트 실력 향상 프로그램	**개발 기간**	2026.07.07 ~ 2026.08.10
개발 인원	6인 (FE · BE · AI · Infra · PM · QA)	**담당 역할**	QA

프로젝트 소개

코드베이스와 기능 명세가 주어지면 사용자가 프롬프트를 입력해 코드를 수정하고, 입력한 프롬프트와 제출 결과에 대한 **LLM 피드백**을 확인하는 프롬프트 평가·학습 프로그램입니다.

QA 프로세스 구축 및 운영

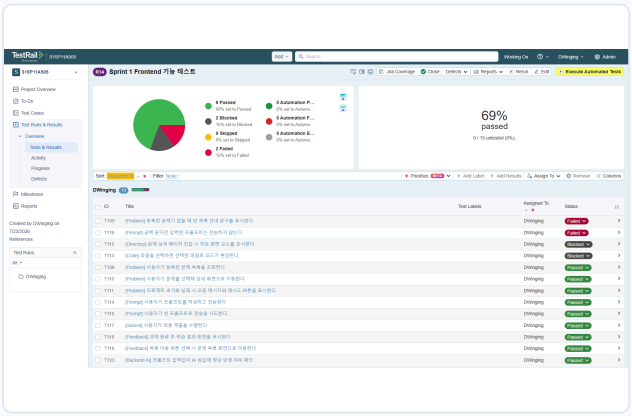
명세 기반 테스트 설계

- 프로토타입 검증 전 기능 명세와 사용자 흐름 검토
- 화면 기능과 백엔드 API를 기준으로 테스트 범위 구분
- TestRail에서 실행 결과와 검증 상태 관리

결함 기록과 개발 협업

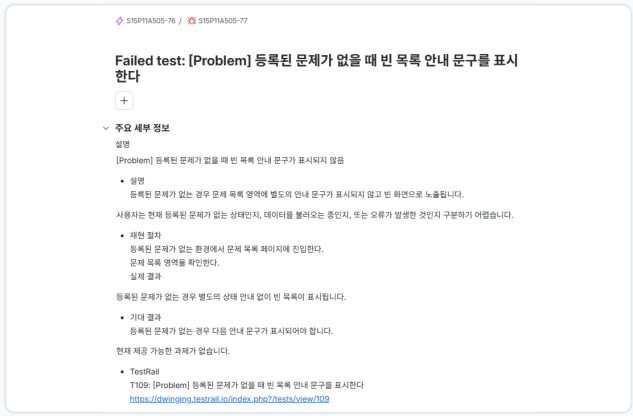
- 실제 결과와 기대 결과의 차이를 근거 중심으로 기록
- 중요 결함은 개발자에게 즉시 공유하고 원인 범위 확인
- 기능 수정이 필요한 항목은 정리 후 팀원들과 회의 진행

QA 산출물



TestRail 테스트 관리

기능별 테스트 케이스와 Pass·Fail·Block 실행 상태를 기록함.



결함 및 변경 요청 기록

재현 조건, 실제 결과, 변경 요청 사항을 정리함.

현재 상태

- 1차 프로토타입 테스트 케이스 13건 수행 완료
- 문제 샘플 데이터가 준비되지 않은 기능 2건은 Block 상태로 기록
- 반복 프롬프트와 다중 파일 관리가 포함된 2차 프로토타입 구축 중

Engineering & Optimization | PCCP Java Lv4 | Baekjoon Platinum II

03. 알고리즘 문제 해결 기반의 코드 최적화 경험

문제 해결을 넘어, 실행 성능과 시스템 동작 원리를 분석하는 엔지니어링 접근

기간	2023.04 ~ 현재	**사용 언어**	Java(주력), Python, Kotlin, C++
플랫폼	Baekjoon · Programmers · SWEA · LeetCode	**핵심 성과**	PCCP Java Lv4 · 삼성 B형 · Baekjoon Platinum II

역량 기술 개요

- 단순히 알고리즘 문제를 해결하는 데 그치지 않고, **실행 성능**과 **시스템 동작 원리**를 분석·검증하는 엔지니어링 접근 방식을 지향합니다.
- 대규모 반복 연산 환경에서 **조건 분기 구조**, 자료구조 접근 방식, 객체 생성 비용 등 런타임에 영향을 주는 요소를 분석합니다.
- **가설 수립 → 코드 개선 → 성능 검증 → 기술 자산화**의 과정을 반복하며 문제 해결 역량을 발전시키고 있습니다.

공학적 접근 방식

- 근거 중심의 문제 해결을 위해 **성능 병목 원인**을 분석하고 가설을 수립한 뒤 실험을 통해 검증합니다.
- 실행 흐름과 자료구조의 동작 원리를 고려하여 **불필요한 연산과 메모리 사용**을 최소화합니다.
- 성능과 가독성의 균형을 고려하여 **Hotspot 구간**에만 선택적으로 최적화를 적용합니다.
- GitHub, log.md, summary.md, Velog를 활용하여 분석 과정과 개선 결과를 **지속적으로 문서화**합니다.

핵심 성과

조건 분기 재설계를 통한 알고리즘 성능 최적화

BOJ 1854번 **K번째 최단 경로** 문제에서 **조건 분기 구조**와 PriorityQueue 접근 방식을 재설계하여 **실행 시간과 메모리 사용량**을 개선했습니다.

항목	개선 전	개선 후	개선 효과
실행 시간	1132ms	596ms	**약 47% 단축**
메모리	63MB	40MB	**약 35% 절감**

- **PriorityQueue 연산 최소화** 및 조건 분기 구조 재설계
- 동일 환경 반복 제출을 통한 **성능 개선 검증**
- 개선 원인과 분석 결과를 **기술 블로그로 문서화**

참고 자료

📁 **GitHub Repository** Algorithm Repository

📝 **Velog Series** 알고리즘 풀이 및 회고 시리즈